EVENTOS NOVA

Producto Integrador de Aprendizaje

Implementación de una solución basada en lógica de negocio

Equipo 16 / FIME / UANL

Integrante	Matrícula
Omar Tadeo Gasca Deciga	2226708
Osmar Abdi Sánchez Santiago	2226699
Gabriela Alejandra Fernández Briones	2226870

Curso: Componentes de Lógica de Negocios

Profesor: Miguel Ángel Gómez Marroquín

Fecha: 6 de noviembre de 2026

Reporte técnico y evidencias de preparación

Aplicación verificada localmente. Publicación del repositorio PIA, ejecución remota de CI/CD y despliegue público pendientes de destino.

Guía de lectura

El reporte integra el análisis refinado, arquitectura, decisiones, evidencia y conclusiones. El proyecto adjunto contiene código, historias Gherkin, BPMN editable, OpenAPI, matrices y documentos fuente.

Parte	Contenido
1	BRD completo refinado y reglas BR01-BR10
2	15 historias con criterios Gherkin y glosario
3	Arquitectura, DDD, UML, estados, SOLID y ADR
4	Trazabilidad, datos y resultados de pruebas
5	Demostración, despliegue, conclusiones y referencias

Alcance de la evidencia

La suite aprobó 269 pruebas y omitió la de PostgreSQL por falta de servicio. La cobertura total de líneas y ramas es 97.39%. Se verificó además el servidor Uvicorn con 20 peticiones HTTP reales. Los 140 casos CSV están incluidos en el total de pruebas. Docker, Actions remoto y producción requieren verificación en el destino real.

Las metas de reducir errores y elevar rentabilidad proceden del BRD. No se presentan como resultados comerciales medidos. Los datos empleados son ficticios.

1. Análisis de negocio

Versión 1.0. Fecha: 06/11/2026. Base: BRD de U2 (04/09/2026) e informe de U3 (17/10/2026). Se mantienen los identificadores originales para trazabilidad. Las decisiones nuevas de implementación se distinguen en los ADR.

1. Propósito y modelo de negocio

Eventos Nova organiza bodas, graduaciones y eventos empresariales. Coordina servicios de salón, alimentos, mobiliario, música, decoración y fotografía contratados con proveedores. Necesita administrar desde la solicitud del cliente hasta la liquidación y cierre, con una única propuesta vigente, responsables identificables y consecuencias económicas trazables.

2. Situación actual y problemas

El Cliente comunica fecha, aforo y tipo de evento. Coordinación cotiza, recibe aceptación y formaliza el contrato. Administración cobra el anticipo. Coordinación solicita disponibilidad y realiza seguimiento. Los cambios por WhatsApp dispersan la información.

ID	Problema	Tratamiento en la solución
P01	Cambios no llegan a todos	Registro de avisos internos con destinatarios
P02	Versiones distintas de una propuesta	Historial de snapshots y versión vigente
P03	Proveedores trabajan con información anterior	Evento CambioAplicado con versión y destinatarios
P04	Cambios autorizados no cobrados	Incremento, cargo y autorización ligados al evento
P05	Seguimiento manual	Estados y temporizador de vencimientos

3. Objetivos e indicadores

OB01: reducir a 0% los incidentes por desincronización atribuida a WhatsApp. OB02: lograr que 90% de proveedores responda en 24 horas. OB03: aumentar 15% el margen global por reducción de fugas de ingresos. OB04: conservar trazabilidad de solicitudes, aprobaciones y ajustes.

Son **metas del negocio**, no resultados comprobados del prototipo. La demo demuestra mecanismos de control, no mide impacto comercial real ni disponibilidad operativa real.

4. Stakeholders y permisos

Cliente: proporciona requisitos, acepta/rechaza propuesta, registra la referencia de contrato y solicita cambios de su evento. Coordinador: prepara y versiona propuestas, asigna proveedores, cotiza cambios, registra adicionales, inicia/finaliza y cierra. Administración: registra pagos, decide excepciones y consulta rentabilidad. Proveedor: responde y cancela sus propias solicitudes. Sistema: identifica solicitudes vencidas y registra alertas.

Cada token identifica una persona o actor concreto. Cliente solo accede a sus eventos. Proveedor solo recibe sus solicitudes y no consulta pagos del cliente. Las decisiones financieras se reservan a Administración.

5. Alcance

Incluye alta del evento, propuesta versionada, aceptación/rechazo, contrato, anticipo, coordinación y sustitución de proveedores, plazos, cambios y autorizaciones, adicionales y cobros, rentabilidad por evento, ejecución, liquidación y cierre con acta. La interfaz de demostración es Swagger UI.

Fuera de alcance: contabilidad general, nómina, recursos humanos, marketing, diseño creativo integral, rutas de transporte, operaciones sin un evento Nova y procesamiento bancario real. Las notificaciones son internas. La firma consiste en el registro autenticado de referencia contractual, sin certificación de firma electrónica.

6. Necesidades

ID	Necesidad	Prioridad
BN01	Comunicación centralizada	Alta
BN02	Propuesta vigente e historial	Alta
BN03	Avisar cambios relevantes	Alta
BN04	Registrar solicitante y autorizador	Alta
BN05	Impacto comercial y rentabilidad	Alta
BN06	Solicitudes, decisiones y plazos de proveedores	Alta
BN07	Anticipos, pagos y saldos	Media
BN08	Proveedores Backup para contingencias	Media

7. Reglas verificables

ID	Regla refinada	Evidencia
BR01	Contrato firmado y anticipo acumulado >=30% del monto firmado para confirmar. No se generan reembolsos del anticipo al cancelar.	Event.sign/pay/cancel
BR02	Respuesta antes de `sent_at + 48h`. En el límite exacto ya está vencida. Temporizador cada 60 s registra la alerta.	SupplierRequest.respond/expire
BR03	Cambios a 15 días naturales o más del evento sin penalización	ChangePolicy.evaluate
BR04	Cambios a 14 días o menos: 15% del incremento, pendientes de decisión de Administración	request_change/apply_change/reject_change
BR05	Registrar el adicional antes de prestarlo	add_extra/deliver_extra
BR06	Ante cancelación/rechazo/vencimiento, registrar sustitución mediante identificador de solicitud anterior y proveedor Backup activo	request_supplier
BR07	Una versión vigente y conservación de todas las anteriores	Proposal.revise, transacción conjunta
BR08	Pago identificado por terminal o transferencia antes de prestar el adicional	pay_extra/deliver_extra
BR09	Cierre con saldo cero, contrato, cambios resueltos, adicionales prestados y acta	CAN_CLOSE y Event.close
BR10	Registrar responsable, instante, condición anterior/nueva y decisión	changes, versions, domain_events

Los límites de catálogo se interpretan como capacidad máxima por línea de cotización. El CSV no define inventario compartido ni un aforo total de recinto. Para disponibilidad de proveedores se impide reservar al mismo proveedor dos veces en una fecha mediante restricción única en DB.

8. Requisitos no funcionales

RNF01: auditoría persistente consultable. RNF02: total vigente consistente con versión. RNF03: autenticación y permisos por identidad. RNF04: health check y despliegue reproducible; no se promete SLA. RNF05: errores y estados comprensibles. RNF06: no cerrar con datos críticos incompletos. RNF07: alertas registradas con destinatarios. RNF08: rollback de evento, propuesta y eventos de dominio si falla la transacción.

Calidad PIA: cobertura >=80% de `src`, pruebas unitarias e integración, estilos verificados, OpenAPI, configuración por entorno, Docker Compose y CI/CD. El estado realmente verificado se registra en `evidencias/RESUMEN.md`.

9. Proceso TO-BE y excepciones

La propuesta inicia en BORRADOR. Coordinación la envía y el Cliente acepta o rechaza. Tras rechazo se prepara nueva versión. La aceptación habilita contrato y pagos; alcanzar el anticipo confirma el evento. Coordinación asigna servicios. Cada proveedor responde dentro del plazo o vence y se inicia contingencia. Los cambios preservan el historial y la autorización. El evento se ejecuta en su fecha con proveedores confirmados y sin cambios pendientes. Se resuelven adicionales, se finaliza, se liquida si falta saldo y se firma el acta de cierre.

Excepciones modeladas: proveedor cancela, rechazo, vencimiento, duplicidad de reserva, aumento de invitados, cambio de horario/menú, recargo tardío, cambio rechazado, adicional sin pago, saldo pendiente y transición repetida o inválida.

10. Ambigüedades y supuestos

El informe U3 §1.4 registra H1-H5 como validados por el equipo aunque el ADR-005 aún figura propuesto. Se aplican: liquidación antes o después del evento; día 15 inclusive sin cargo; cálculo automático y autorización humana; 15% sobre incremento; adicional sin pago no prestado. El PIA implementa también las extensiones de ejecución y cierre.

Nuevas decisiones: cambios del Cliente requieren revisión comercial incluso dentro de plazo; los proveedores de demo son datos complementarios; la fecha de evento para operaciones CSV se deriva de solicitud +60 días; el precio del cambio lo cotiza manualmente un responsable. No existe fórmula de precio por invitado en las fuentes. La aprobación no acredita una validación por el profesor ni un stakeholder externo.

11. Éxito y aceptación

Cada US01-US15 debe tener criterio Gherkin, componente y prueba identificables. La demostración debe mostrar un flujo completo y rechazos verificables. Se acepta la entrega técnica cuando las pruebas, cobertura, documentación, repositorio y despliegue coinciden con la versión presentada. Las metas OB01-OB03 requieren medición en operación real.

12. Conclusión

La lógica central conserva una versión coherente del evento y permite relacionar autorización, cobro y prestación del servicio. El prototipo hace verificables las reglas del BRD y mantiene explícitas las decisiones que las fuentes no definen, facilitando su revisión por el equipo.

2. Historias y aceptación

Se conservan US01-US15 de U2 y se refinan conforme a H1-H5 de U3. Cada criterio tiene prueba pytest identificada en TRAZABILIDAD.csv. El archivo `aceptacion.feature` conserva el formato Gherkin; las pruebas ejecutables son las de pytest.

US01

Como Coordinador, quiero registrar datos base y servicios, para preparar una propuesta V1.0.

Reglas: BN02, BR07. Componente: `Event, Proposal.revise`. API: `POST /events`. Prueba: `test_US07_revision_history_and_duplicate_accept`.

```
@US01
Escenario: US01 criterio 1
  Dado un cliente activo, fecha futura y servicios habilitados
  Cuando el Coordinador registra el evento
  Entonces se conserva el evento BORRADOR y la propuesta V1.0 con el total decimal
@US01
Escenario: US01 criterio 2
  Dado un aforo de cero
  Cuando se intenta registrar el evento
  Entonces la API responde 422 y no persiste el alta
```

US02

Como Cliente, quiero aceptar la propuesta enviada, para iniciar la contratación.

Reglas: BR07. Componente: `Proposal.accept, Event.accept`. API: `POST /events/{id}/accept`. Prueba: `test_US07_revision_history_and_duplicate_accept`.

```
@US02
Escenario: US02 criterio 1
  Dado una propuesta ENVIADA del Cliente autenticado
  Cuando el Cliente acepta
  Entonces la propuesta queda ACEPTADA y el evento ACEPTADO
@US02
Escenario: US02 criterio 2
  Dado una propuesta ya aceptada
  Cuando el Cliente vuelve a aceptar
  Entonces la API responde 409
```

US03

Como Administración, quiero registrar el anticipo del 30%, para confirmar el evento.

Reglas: BR01. Componente: `Event.pay`. API: `POST /events/{id}/pay`. Prueba: `test_BR01_30_percent_boundary`.

```
@US03
Escenario: US03 criterio 1
  Dado un contrato firmado por 1000.00 MXN
  Cuando Administración registra 300.00 MXN
  Entonces el evento pasa a CONFIRMADO
@US03
Escenario: US03 criterio 2
  Dado un contrato firmado por 1000.00 MXN
  Cuando se registran 299.99 MXN
  Entonces el evento permanece ACEPTADO
```

US04

Como Coordinador, quiero solicitar disponibilidad de servicios, para coordinar proveedores.

Reglas: BN03, BN06. Componente: `SupplierRequest, NovaService.request_supplier`. API: `POST /events/{id}/requests`. Prueba: `test_US01_US15_full_business_cycle`.

```
@US04
Escenario: US04 criterio 1
  Dado un evento CONFIRMADO y proveedor activo
  Cuando Coordinación solicita el servicio
  Entonces se crea solicitud PENDIENTE y evento con destinatario y vencimiento
@US04
Escenario: US04 criterio 2
  Dado un evento BORRADOR
  Cuando Coordinación intenta asignar proveedor
  Entonces la API responde 409
```

US05

Como Proveedor, quiero aceptar o rechazar mi solicitud dentro del plazo, para resolver mi participación.

Reglas: BR02. Componente: `SupplierRequest.respond`. API: `POST /requests/{id}/respond`. Prueba: `test_BR02_continuous_hours`.

```
@US05
Escenario: US05 criterio 1
  Dado una solicitud PENDIENTE de mi identidad y menos de 48 horas
  Cuando respondo que acepto
  Entonces la solicitud pasa a CONFIRMADO y se reserva la fecha
@US05
Escenario: US05 criterio 2
  Dado 48 horas exactas desde el envío
  Cuando intento aceptar
  Entonces la API responde 422 por vencimiento
```

US06

Como Sistema, quiero detectar solicitudes sin respuesta, para avisar a Coordinación.

Reglas: BR02, RNF07. Componente: `SupplierRequest.expire, expiry_loop`. API: `Temporizador y POST /requests/expire`. Prueba: `test_US06_48_hours_and_expiration_idempotence`.

```
@US06
Escenario: US06 criterio 1
  Dado una solicitud PENDIENTE con 48 horas cumplidas
  Cuando el temporizador ejecuta su revisión
  Entonces la solicitud pasa a EXPIRADO y queda alerta interna a Coordinación
@US06
Escenario: US06 criterio 2
  Dado una solicitud ya expirada
  Cuando se revisa nuevamente
  Entonces no se duplica la alerta
```

US07

Como Coordinador, quiero generar nuevas versiones de propuesta, para conservar el historial.

Reglas: BR07, BR10. Componente: `Proposal.revise`. API: `POST /events/{id}/revise`. Prueba: `test_US07_revision_history_and_duplicate_accept`.

```
@US07
Escenario: US07 criterio 1
  Dado una propuesta BORRADOR V1.0
  Cuando Coordinación cambia servicios con motivo
  Entonces se agrega V1.1 sin modificar V1.0
@US07
Escenario: US07 criterio 2
  Dado una propuesta ENVIADA
  Cuando se intenta editar directamente
  Entonces se rechaza la modificación hasta resolver la propuesta
```

US08

Como Proveedor, quiero recibir el aviso interno de cambios aprobados, para usar la información vigente.

Reglas: RNF07, RNF08. Componente: `CambioAplicado, NovaService.apply_change`. API: `POST /events/{id}/change y GET /events/{id}/audit`. Prueba: `test_US08_US10_changes_authorization_and_notification`.

```
@US08
Escenario: US08 criterio 1
  Dado un proveedor asignado al evento
  Cuando se aplica un cambio de horario o menú
  Entonces CambioAplicado conserva la nueva versión y el proveedor como destinatario
@US08
Escenario: US08 criterio 2
  Dado un error antes del commit
  Cuando falla la actualización de propuesta
  Entonces evento, propuesta y notificación se revierten juntos
```

US09

Como Cliente, quiero solicitar cambios de aforo con anticipación, para actualizar la cotización sin recargo.

Reglas: BR03. Componente: `ChangePolicy.evaluate, Event.request_change`. API: `POST /events/{id}/change`. Prueba: `test_client_change_waits_for_commercial_review`.

```
@US09
Escenario: US09 criterio 1
  Dado un evento a 15 días naturales o más
  Cuando el Cliente solicita aumentar invitados
  Entonces se calcula recargo 0.00 y espera revisión comercial
@US09
Escenario: US09 criterio 2
  Dado una cotización revisada de un cambio dentro de plazo
  Cuando Administración la autoriza
  Entonces se actualizan aforo y versión sin penalización
```

US10

Como Administración, quiero autorizar cambios tardíos con recargo, para proteger el margen.

Reglas: BR04. Componente: `ChangePolicy, Event.apply_change/reject_change`. API: `POST /events/{id}/approve y reject_change`. Prueba: `test_US08_US10_changes_authorization_and_notification`.

```
@US10
Escenario: US10 criterio 1
  Dado un evento a 14 días con incremento de 100.00 MXN
  Cuando se solicita el cambio
  Entonces queda PENDIENTE con cargo calculado de 15.00 MXN sin cambiar aún el total
@US10
Escenario: US10 criterio 2
  Dado un cambio tardío pendiente
  Cuando Administración autoriza
  Entonces se aplican incremento y cargo con responsable y nueva versión
```

US11

Como Coordinador, quiero gestionar una sustitución Backup, para mantener continuidad tras una cancelación.

Reglas: BR06, BN08. Componente: `SupplierRequest.cancel, request_supplier`. API: `POST /requests/{id}/cancel y POST /events/{id}/requests`. Prueba: `test_US11_backup_replacement_and_booking`.

```
@US11
Escenario: US11 criterio 1
  Dado un proveedor CONFIRMADO
  Cuando el Proveedor cancela con motivo
  Entonces se libera su reserva y se devuelve la lista Backup activa
@US11
Escenario: US11 criterio 2
  Dado una solicitud cancelada y proveedor Backup activo
  Cuando Coordinación registra su sustitución
  Entonces la nueva solicitud conserva el identificador de la anterior
```

US12

Como Administración, quiero consultar ingresos y costo de proveedores, para evaluar el margen por evento.

Reglas: BN05. Componente: `NovaService.profitability`. API: `GET /events/{id}/profitability`. Prueba: `test_US01_US15_full_business_cycle`.

```
@US12
Escenario: US12 criterio 1
  Dado un total de 5990.00 y proveedores por 1000.00 MXN
  Cuando Administración consulta rentabilidad
  Entonces obtiene margen de 4990.00 MXN e importes cobrados por separado
@US12
Escenario: US12 criterio 2
  Dado un Cliente autenticado
  Cuando intenta consultar la rentabilidad
  Entonces la API responde 403
```

US13

Como Coordinador, quiero registrar y cobrar servicios adicionales, para evitar prestación sin control comercial.

Reglas: BR05, BR08. Componente: `Event.add_extra/pay_extra/deliver_extra`. API: `POST /events/{id}/extra, pay_extra, deliver_extra`. Prueba: `test_BR05_BR08_extra_requires_payment_and_trace`.

```
@US13
Escenario: US13 criterio 1
  Dado un evento EN_EJECUCION y adicional registrado
  Cuando Administración registra su pago específico por terminal
  Entonces el adicional queda pagado y puede prestarse
@US13
Escenario: US13 criterio 2
  Dado un adicional sin pago
  Cuando Coordinación intenta marcarlo prestado
  Entonces la API responde 422
```

US14

Como Administración, quiero liquidar el saldo del evento, para resolver la obligación financiera.

Reglas: BR09, H1 U3. Componente: `Event.pay, balance`. API: `POST /events/{id}/pay`. Prueba: `test_post_event_liquidation_follows_U3_H1`.

```
@US14
Escenario: US14 criterio 1
  Dado un evento FINALIZADO con saldo de 4123.00 MXN
  Cuando Administración registra 4123.00 MXN
  Entonces el saldo queda en 0.00 sin duplicar el anticipo
@US14
Escenario: US14 criterio 2
  Dado un saldo de 4123.00 MXN
  Cuando se intenta registrar un pago de 4123.01 MXN
  Entonces la API responde 422
```

US15

Como Coordinador, quiero cerrar el evento con acta, para concluir su ciclo.

Reglas: BR09, BR10. Componente: `CAN_CLOSE, Event.close`. API: `POST /events/{id}/close`. Prueba: `test_US01_US15_full_business_cycle`.

```
@US15
Escenario: US15 criterio 1
  Dado un evento FINALIZADO sin saldo ni pendientes
  Cuando Coordinación registra el acta de cierre
  Entonces el estado queda CERRADO con fecha y responsable
@US15
Escenario: US15 criterio 2
  Dado un evento con saldo pendiente
  Cuando se intenta cerrar
  Entonces la API responde 422 y conserva su estado
```

3. Glosario

Los términos 1-20 proceden del glosario U2. Se aclara su traducción al prototipo y se agregan términos de U3/PIA.

N	Término	Definición	Código / alcance
1	Acta de cierre	Registro formal de conclusión con responsable, fecha y referencia.	Event.closure
2	Administración	Rol responsable de pagos y autorización comercial.	Principal.role ADMIN
3	Anticipo	30% acumulado del monto firmado que habilita confirmación.	Event.pay
4	Backup de Respuesta Rápida	Proveedor activo marcado para contingencias.	supplier.backup
5	Cargo por urgencia	15% del incremento cuando faltan menos de 15 días.	ChangePolicy
6	Cliente	Identidad dueña del evento y propuesta.	Event.client_id
7	Contrato	Referencia autenticada de firma y monto contratado.	Event.contract
8	Coordinador	Responsable operativo de propuesta y logística.	Principal.role COORDINADOR
9	Cotización / propuesta	Agregado con estado y versiones de servicios e importes.	Proposal
10	Ejecución del evento	Estado operativo del día programado.	EN_EJECUCION
11	Estado de proveedor	Pendiente, confirmado, rechazado, expirado o cancelado.	SupplierRequest.state
12	Fecha límite	15 días naturales inclusive sin cargo.	ChangePolicy.free_days
13	Gateway de aprobación	Decisión BPMN que representa autorización.	Modelo BPMN, no entidad
14	Liquidación	Pago del saldo restante, antes o después de ejecución según H1.	Event.balance
15	Plataforma centralizada	API, persistencia y registro interno de avisos.	Sistema completo

N	Término	Definición	Código / alcance
16	Proveedor Categoría A	Clasificación de cumplimiento de la fuente, sin regla automatizada en este PIA.	Fuera del modelo funcional
17	Rentabilidad	Ingreso cotizado menos costo de proveedores confirmados vigentes.	profitability
18	Servicio adicional	Prestación añadida, registrada y pagada antes de entregarse.	Event.extras
19	Tipo de evento	Boda, graduación o empresarial; se conserva como descripción en esta demo.	title / descripción de catálogo
20	Versionado	Conservar V1.0, V1.1 y siguientes sin sobreescribir las anteriores.	Proposal.versions
21	Solicitud de cambio	Intención de modificar aforo, menú u horario con impacto comercial.	Event.changes
22	Cambio pendiente	Solicitud que espera decisión de Administración.	PENDIENTE
23	Solicitud de proveedor	Petición independiente con plazo continuo de 48 horas.	SupplierRequest
24	Aviso interno	Evento persistido con destinatarios, visible tras commit.	domain_events
25	Unidad de trabajo	Frontera transaccional que confirma o revierte cambios relacionados.	SqlUnitOfWork
26	Versión de concurrencia	Número usado para detectar escrituras basadas en datos obsoletos.	aggregates.version

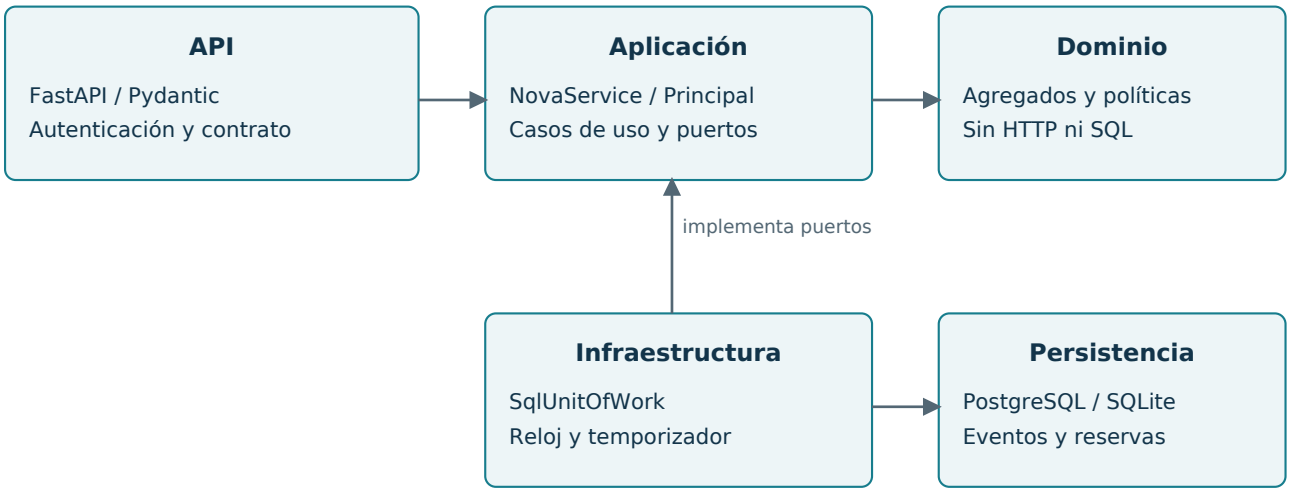
4. Arquitectura e implementación

Continuidad con U3

Se mantienen los tres agregados del diseño: Propuesta, Evento y SolicitudDeProveedor. Los nombres Python son `Proposal`, `Event` y `SupplierRequest`. Se conserva el dominio independiente de FastAPI y SQLAlchemy, la unidad de trabajo y la concurrencia optimista. El PIA implementa también las extensiones US11-US15.

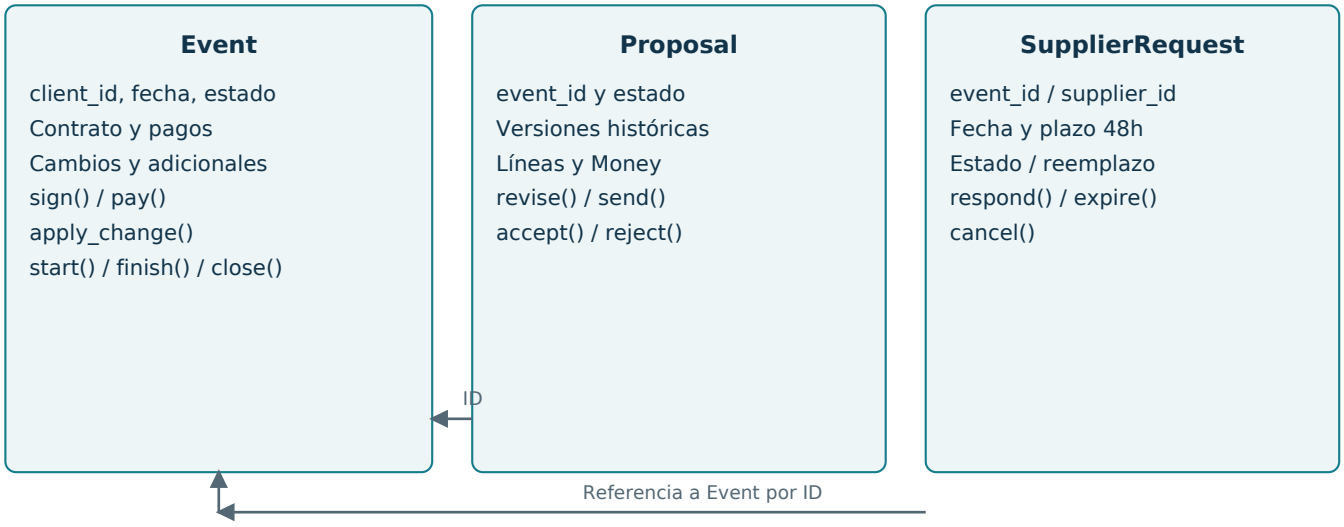
Las diferencias con el diseño teórico están registradas en ADR-006 a ADR-009. El documento describe el código existente, no clases pendientes como si estuvieran implementadas.

Capas y dependencias



El punto de composición está en `src/api/main.py`. Los casos de uso reciben la fábrica de unidad de trabajo y el reloj. El dominio importa solo biblioteca estándar y módulos de su misma capa. El temporizador es un adaptador que invoca el caso de uso de vencimiento.

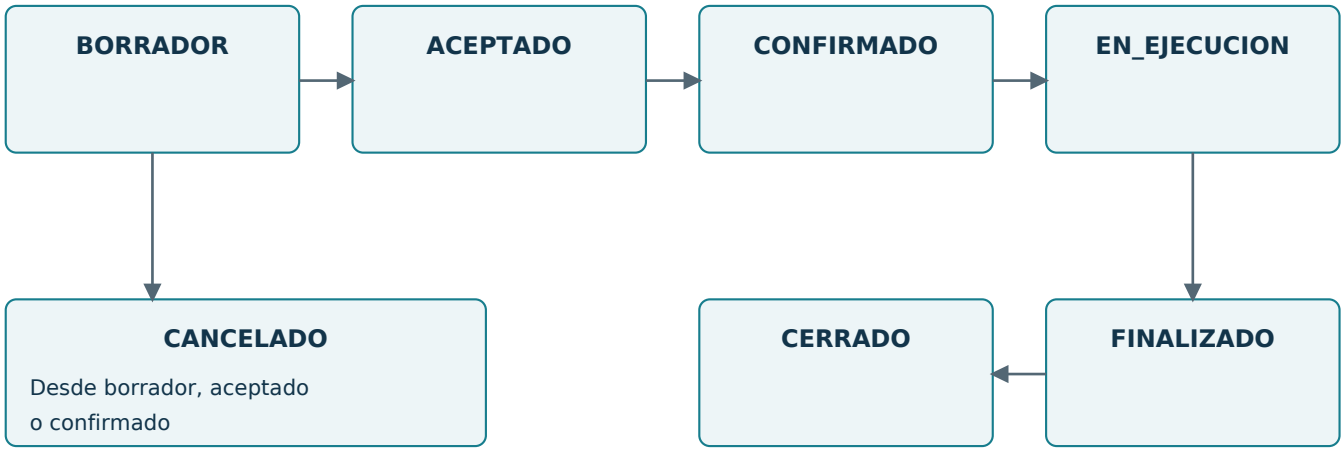
DDD y fronteras



Event mantiene contrato, pagos, cambios y adicionales en su frontera de consistencia. Proposal mantiene snapshots históricos: current devuelve copia y revise agrega, nunca elimina ni reemplaza versiones. SupplierRequest tiene su propio plazo y resolución. Cliente, recurso y proveedor son catálogos simples.

Se usan Money y Line como Value Objects inmutables. Aforo y fecha se validan dentro de Event, en vez de crear clases vacías. Versiones, pagos y contratos se serializan como datos dentro de sus agregados. Esta simplificación se registra expresamente respecto a U3.

Estados e invariantes



Propuesta: BORRADOR -> ENVIADA -> ACEPTADA; rechazo de ENVIADA vuelve a BORRADOR para cotizar de nuevo. Solicitud: PENDIENTE -> CONFIRMADO/RECHAZADO/EXPIRADO; CONFIRMADO -> CANCELADO. Cambio: PENDIENTE -> APLICADO/RECHAZADO. Liquidado es una condición financiera (balance=0), no un estado que impida la ejecución o cierre.

Invariantes: importes finitos no negativos, cantidades/aforo enteros positivos; total consistente con versión; contrato antes de pagos; confirmación con >=30%; respuesta antes de 48h; recargo solo al aplicar el cambio aprobado; adicional pagado antes de prestarse; cierre sin deuda ni información crítica pendiente. Los endpoints no permiten fijar arbitrariamente el estado.

Specifications

CAN_SEND = ActiveClient & HasLines & Authorized implementa precondiciones de envío. CAN_CLOSE = ~(PendingBalance | MissingInformation) protege BR09. AND, OR y NOT intervienen en reglas reales. El contexto Authorized=True en el envío es consecuencia del permiso de Coordinación/Administración ya verificado. El flag CSV se usa solo en los casos diagnósticos.

Persistencia y eventos

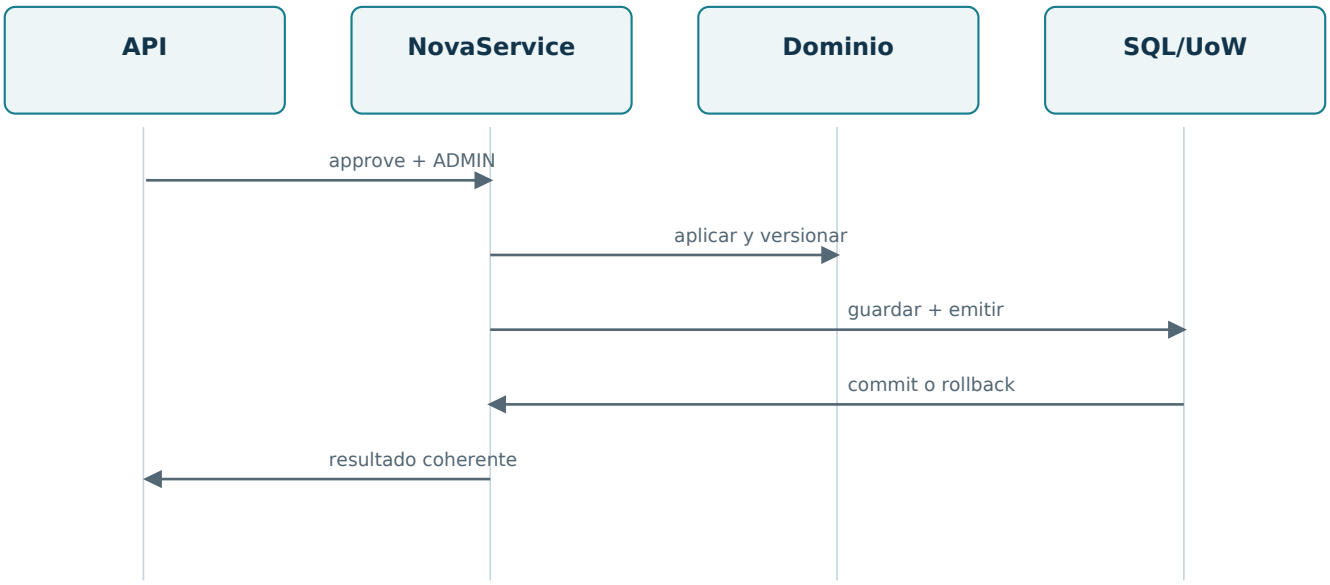
SQLAlchemy Core crea tres tablas: aggregates(kind,id,data,version), domain_events(id,event_id,name,actor,at,payload) y supplier_bookings(supplier_id,day,request_id). Los agregados se guardan como JSON con identidad compuesta y versión. El formato conserva importes decimales como texto. Las referencias se validan en aplicación. No se afirma que exista una FK relacional dentro del JSON.

La escritura comprueba la versión previamente leída. Si otra transacción la modificó, responde 409 sin sobrescribirla. La clave única de proveedor/fecha impide doble reserva. La transacción revierte todos los cambios ante error. Alta y modificaciones de cotización actualizan Evento y Propuesta conjuntamente;

aceptación también sincroniza ambos; respuesta/cancelación de proveedor sincroniza solicitud, costo y reserva.

Los eventos de dominio se registran atómicamente en `domain_events`, con nombre, responsable, fecha y payload. Un lector solo los ve tras commit. Constituyen auditoría y avisos internos. No hay broker, correo ni promesa de entrega externa. Si se añade correo, se necesita un consumidor con reintentos e idempotencia; esta entrega no lo simula.

Secuencia de cambio tardío



Patrones y SOLID

Decisión	Aplicación comprobable	Consecuencia
Domain Model	Event.pay, SupplierRequest.respond, ChangePolicy.evaluate	Reglas probables sin HTTP ni DB
Specification	ActiveClient, HasLines, Authorized, PendingBalance, MissingInformation	Reglas combinables y comprobables por separado
State Machine	EVENT_TRANSITIONS y métodos de dominio	Cambios de estado explícitos, tabla completa probada
Repository/Unit of Work	SqlUnitOfWork.get/save/commit	Persistencia y transacciones fuera del dominio
Registro de Domain Events	emit dentro de la misma transacción	Auditoría consistente con cambios persistidos
CQRS básico	Métodos de lectura separados de execute	Una DB y snapshots compartidos, sin bus ni event sourcing

SRP: dominio protege invariantes, aplicación coordina, infraestructura persiste, API valida contrato HTTP. NovaService agrupa casos de uso y es un punto que convendría separar por módulo si crece. OCP: las Specifications se componen y los relojes/adaptadores se inyectan; el registro de comandos aún exige editar la tabla de rutas. LSP: SqlUnitOfWork cumple los métodos del puerto y se prueba contra DB; el reloj fijo implementa el mismo contrato que SystemClock. ISP: Clock tiene solo now, UnitOfWork solo operaciones necesarias. DIP: dominio no depende de framework; aplicación recibe puertos y el arranque elige

adaptadores.

Límites operativos

Una instancia y un worker de Uvicorn para la demo. El temporizador registra vencimientos con hasta un intervalo adicional de 60 s, pero la aceptación a 48h exactas siempre se rechaza. No hay sistema de cuentas/login, OAuth, envío de correo, cobro bancario, certificado de firma o inventario multialmacén. Los tokens se administran por entorno. La publicación, ejecución real de Compose y el CI remoto deben comprobarse en el destino elegido, sin atribuirles evidencia local.

5. Decisiones técnicas

Los ADR-001 a ADR-005 preservan las decisiones U3 con su estado histórico de propuesta. La implementación adopta sus fundamentos. Los ADR-006 a ADR-009 registran refinamientos implementados en esta entrega y pendientes de revisión del equipo.

ADR-001 / Domain Model

Contexto: BR01-BR04, BR07 y BR09 combinan dinero, plazos y estados. Decisión: tres agregados con comportamiento. Alternativas: Transaction Script y Table Module. Consecuencias: mayor estructura inicial y pruebas de reglas independientes; evitar trasladar descuentos de las tiendas del laboratorio.

ADR-002 / Frontera de Evento

Contexto: contrato, anticipo y saldo deben ser consistentes de inmediato. Decisión: pagos, cambios y adicionales dentro de Event; Propuesta y SolicitudDeProveedor separados por ID. Alternativa: cuenta separada con consistencia eventual. Consecuencias: Event crece y puede tener conflictos de escritura, mitigados mediante versiones.

ADR-003 / Unidad de trabajo y eventos

Contexto: RNF08 y BR07 prohíben versiones contradictorias. Decisión U3: una transacción por agregado salvo alta/cambio de Evento y Propuesta; eventos visibles después del commit. Alternativa: consistencia eventual. Consecuencias: transacciones más amplias. El PIA extiende la atomicidad a aceptación y respuestas de proveedor con costo/reserva para no dejar estados parciales. Los avisos son registros internos transaccionales; no hay publicación a broker.

ADR-004 / CQRS básico

Contexto: propuesta vigente, solicitudes, saldo y margen tienen consultas distintas. Decisión: lecturas y comandos separados en NovaService, una sola DB. Alternativa: event sourcing o bases separadas. Consecuencias: menor complejidad; algunas vistas reutilizan el agregado para calcular saldo, simplificación respecto al diseño U3 de lectores SQL separados.

ADR-005 / H1-H5

Contexto: contradicciones U2 sobre día 15, autorización, base del cargo, liquidación y adicional. Decisión: seguir la tabla §1.4 del informe U3: 15 inclusive sin cargo, 15% sobre incremento, autorización administrativa tardía, liquidación previa o posterior, adicional con pago antes de prestación. Alternativa: bloquear avance hasta aclaración externa. Consecuencias: política explícita y localizable. El estado histórico del ADR es propuesta, mientras el informe marca esas resoluciones validadas por el equipo; no se presume aprobación del profesor.

ADR-006 / Python, FastAPI y snapshots SQL

Contexto: U4 y U5 usan Python, FastAPI, pytest y SQLAlchemy. Decisión implementada: continuar esa tecnología, dominio dataclass, SQLAlchemy Core y JSON por agregado en PostgreSQL/SQLite. Alternativas: ORM con una tabla por entidad de U3 o reescritura en otro lenguaje. Consecuencias: puesta en marcha y transacciones simples, menos joins, restricciones referenciales internas en aplicación y necesidad de migrar explícitamente el formato JSON cuando evolucione. No existe una tabla por cada Value Object. PostgreSQL se configura en Compose/CI; SQLite se verifica localmente.

ADR-007 / Seguridad y cotización comercial

Contexto: RNF03 requiere roles asociados a personas y el CSV solo trae una bandera genérica. Decisión implementada: tokens Bearer por identidad desde entorno, propiedad de cliente/proveedor, importes validados. Los cambios solicitados por Cliente esperan Administración; Coordinación aplica los no tardíos. El incremento se cotiza manualmente y se registra. Alternativas: confiar en un encabezado X-Role, aceptar roles en el body o implementar OAuth completo. Consecuencias: apropiado para demostración sin suplantación trivial del rol en el cuerpo; se necesita gestión de identidades y rotación de tokens para una operación comercial real.

ADR-008 / Tiempo, estados y servicios adicionales

Contexto: BR02, BR03, BR08 y H1 necesitan fronteras precisas. Decisión implementada: instantes UTC, días naturales en America/Mexico_City, respuesta de proveedor estrictamente antes de 48h, inicio en la fecha del evento sin cambios pendientes, liquidación general flexible y cierre con saldo cero. El pago específico del adicional exige terminal/transferencia, aunque el pago general admite efectivo. Alternativa: usar fecha UTC para el límite o bloquear ejecución por deuda. Consecuencias: pruebas de medianoche y bordes; vencimiento revisado por temporizador con intervalo configurable y una sola instancia.

ADR-009 / Evidencias y publicación

Contexto: la rúbrica exige CI/CD activo, Docker y URL funcional, pero preparar archivos localmente no demuestra ejecuciones remotas. Decisión implementada: generar evidencias reales de pytest/Ruff/API, workflow y despliegue configurables, e identificar pendientes de plataforma. Alternativa: incluir capturas o URLs de ejemplo como si fueran reales. Consecuencias: entrega revisable y reproducible; falta incorporar el repositorio final y comprobar Actions/Compose/producción en un destino real. El historial asistido no suplanta contribuciones del equipo.

6. Proceso BPMN

El archivo `architecture/eventos_nova.bpmn` es BPMN 2.0 editable con cuatro carriles: Cliente, Coordinación, Administración y Proveedor. Se abre con un modelador BPMN compatible. Contiene proceso principal, rechazo de propuesta, validación de anticipo, solicitudes, temporizador de 48 horas, rechazo/vencimiento, contingencia Backup, cambios opcionales, adicionales, liquidación y cierre.

El temporizador PT48H es un boundary event que interrumpe la espera de respuesta. En el código corresponde a `SupplierRequest.expire` y al adaptador `expiry_loop`. Los gateways de aceptación, anticipo, proveedor y saldo conservan alternativas explícitas. `Gestionar cambios` y `Resolver adicionales` son subprocesos colapsados en el plano principal y contienen sus nodos y secuencias en el XML. Los archivos SVG complementarios permiten consultar su detalle. Los data objects representan propuesta y contrato.

El BPMN describe el proceso de negocio, no un motor de ejecución. `isExecutable=false` indica que FastAPI y el modelo de dominio implementan sus reglas. No se afirma que el XML se despliegue en un motor BPMN.

Refinamientos frente a U2: rechazo vuelve a cotización, borde temporal explícito, cambio tardío sujeto a autorización, historial de versión, pago del adicional previo a prestación y liquidación general compatible con H1 de U3. En una operación real las actividades no se bloquean esperando un correo: el estado persiste entre peticiones y el proveedor responde de forma independiente.

La integridad de IDs y referencias de secuencia fue verificada localmente. La rúbrica debe evaluarse con el XML, las historias, el código y las pruebas de la matriz. El PDF BPMN original de U2 también se conserva en `docs/fuentes`.

7. Adaptación de datos

Los seis CSV del paquete `16_Eventos_Nova/01_Alumno/datos` se conservan byte a byte en `data/original`. Contienen 40 actores, 20 recursos, 60 operaciones, 117 líneas, 120 casos y 20 transiciones. No se usaron respuestas reservadas al docente.

Elemento fuente	Tratamiento	Justificación
actor_id y nombre Organizador	Referencia de Cliente de demo, conservando ID/nombre/activo/ciudad	El CSV usa actor genérico y no contiene cliente real
es_preferente, promocion_activa	Conservados en fuente, excluidos del dominio	El BRD Nova no define descuentos VIP
autorizado	Solo diagnóstico de Specifications	La API autentica por token, no por esta bandera
COTIZADO	BORRADOR de Evento y Propuesta V1.0	U2/U3 exigen propuesta inicial, no flujo genérico CONTRATADO/REALIZADO
fecha_solicitud	Se conserva en la versión importada	No se presenta como fecha del evento
fecha de evento ausente	solicitud +60 días para cada fixture	Dato derivado, declarado, reproducible
costo proveedor ausente	Se registra al asignar, proveedores complementarios PROV-001/002/003	No se infiere rentabilidad desde precio de venta
EVE-T001..060	Precondiciones AND: activo, detalle y autorización, solo diagnóstico	No sustituye pruebas de API y permisos
EVE-B001..060, regla VIP	Adaptados a BR01 de contrato y anticipo del 30%	El paquete exige adaptar oráculos al BRD propio
EVE-S001..020	COTIZADO se adapta a BORRADOR. aprobar=aceptar propuesta; completar=ciclo hasta cierre; cancelar=cancelación; repetir_aprobacion/saltar_estado=conflicto	La máquina real deriva de U2/U3

En las transiciones `completar` el ensayo de dominio aporta contrato, pago total, fecha y proveedor confirmado sintéticos. Esto se declara en la matriz y no se confunde con un recorrido HTTP. Las pruebas de integración hacen el recorrido de API con persistencia real.

Los resultados esperados se calculan con un oráculo independiente en centavos enteros para el anticipo y tablas explícitas de estados. `scripts/evaluate_csv.py` usa el código de dominio para lo observado. Pytest parametriza y compara las 140 filas. Se exportan resultados reales/esperados, identificación de la adaptación y PASS/FAIL.

Los fixtures pueden conservar clientes inactivos o recursos no habilitados. El importador no borra esos casos adversos. Las altas por API aplican validación estricta de actividad, capacidades, cantidades y referencias.

U4 aporta el patrón de Specifications, Money/Decimal, estados y endpoints FastAPI. U5 aporta casos parametrizados, separación de pruebas, commit/rollback, seguridad y CI. Sus descuentos, límites de unidades y estados de pedidos no son reglas del PIA.

8. Calidad y evidencia

Fecha: 06/11/2026. Python 3.12.14, Linux.

Comprobación	Resultado real	Evidencia
Suite pytest	269 aprobadas, 1 omitida, 0 fallos	pytest.log y junit.xml
Cobertura de src, líneas y ramas	97.39%, umbral 80%	coverage.json, coverage.xml, htmlcov/index.html
Líneas ejecutadas	870/882	coverage.json
Ramas ejecutadas	211/228	coverage.json
Casos CSV	120 comparados, 0 diferencias	resultados_120_casos.csv
Transiciones CSV	20 comparadas, 0 diferencias	resultados_20_transiciones.csv
API con Uvicorn y HTTP real	20 peticiones, todas con código esperado	demo_http.json
Ruff check y format --check	PASS	ruff.log
Persistencia SQLite	commit, reapertura, rollback y CAS aprobados	pruebas y junit.xml
PostgreSQL	NO EJECUTADO: servicio no disponible	prueba marcada SKIP
Docker Compose	Configuración preparada, no ejecutada en este entorno	docker-compose.yml
GitHub Actions	Workflow preparado, sin ejecución remota del PIA	.github/workflows/ci.yml
URL pública	Pendiente de repositorio/plataforma	DESPLIEGUE.md

Los 140 ensayos CSV forman parte de las 269 pruebas aprobadas; no se suman otra vez al total. La prueba de HTTP externo sí se ejecutó contra un servidor real. Las pruebas unitarias e integración usan además TestClient y SQLite. La advertencia de deprecación registrada procede de Starlette/AnyIO y no afectó el resultado.

La evidencia HTTP registra roles, rutas, códigos y datos ficticios. Excluye las cabeceras de autenticación. Se verificaron rechazo 409 por aceptación duplicada, 422 por adicional sin pago, 422 por cierre con saldo y cierre final correcto.

La cobertura no demuestra ejecución de todas las plataformas ni acredita mejoras comerciales. No hay capturas simuladas de Actions, Docker o producción.

9. Trazabilidad de aceptación

La matriz completa con función de prueba y ruta está en docs/TRAZABILIDAD.csv. Esta vista relaciona las 15 historias con reglas y componentes implementados.

Historia	Reglas	Componente
US01	BN02, BR07	Event, Proposal.revise
US02	BR07	Proposal.accept, Event.accept
US03	BR01	Event.pay
US04	BN03, BN06	SupplierRequest, NovaService.request_supplier
US05	BR02	SupplierRequest.respond
US06	BR02, RNF07	SupplierRequest.expire, expiry_loop
US07	BR07, BR10	Proposal.revise
US08	RNF07, RNF08	CambioAplicado, NovaService.apply_change
US09	BR03	ChangePolicy.evaluate, Event.request_change
US10	BR04	ChangePolicy, Event.apply_change/reject_change
US11	BR06, BN08	SupplierRequest.cancel, request_supplier
US12	BN05	NovaService.profitability
US13	BR05, BR08	Event.add_extra/pay_extra/deliver_extra
US14	BR09, H1 U3	Event.pay, balance
US15	BR09, BR10	CAN_CLOSE, Event.close

10. Demostración y defensa

Preparación

Usa una base ficticia recién inicializada. Inicia la aplicación y abre `/docs`. Cambia el token de Authorize según el rol indicado. Elige un ID nuevo, por ejemplo DEMO-PIA-01. El proveedor de demo solo puede aceptar un evento por fecha; para repetir el recorrido completo prepara otra base de demo o usa una fecha y proveedor disponibles.

Recorrido en vivo de 8 a 12 minutos

1. **Coordinación:** `POST /events`, cliente EVE-C001, fecha de hoy en México, 100 invitados, recurso EVE-P001 y cantidad 10. Total 5890.00 MXN, versión V1.0.

2. **Coordinación:** send con `{}`. **Cliente:** `accept` con `{}`. Repetir accept demuestra 409.

3. **Cliente:** `sign`, referencia CONTRATO-01. **Administración:** pay, monto 1767.00, referencia ANTICIPO-01, método TRANSFERENCIA. Se confirma el evento.

4. **Coordinación:** crear solicitud en `/events/{id}/requests`: proveedor PROV-001, servicio Banquete, costo 1000.00. **Proveedor PROV-001:** responder accepted=true en `/requests/{request_id}/respond`.

5. **Coordinación:** change con guests=110 e increment=100.00. Al ser hoy, queda pendiente con cargo 15.00. **Administración:** `approve` con change_id. Total 6005.00, nueva versión conservando V1.0. Consultar audit muestra condición anterior, nueva y destinatarios.

6. **Coordinación:** `start`. Registrar `extra`, descripción Música, importe 100.00. Intentar `deliver_extra` sin pago demuestra 422. **Administración:** pay_extra, extra_id, referencia EXTRA-01 y método TERMINAL. **Coordinación:** `deliver_extra` y `finish`.

7. **Coordinación:** intentar `close` demuestra bloqueo por saldo. **Administración:** pagar el saldo informado por la API. **Coordinación:** cerrar con acta ACTA-01. Estado CERRADO, saldo 0.00.

8. **Administración:** consultar profitability. Ingreso 6105.00, costo proveedor 1000.00, margen 5105.00 MXN. Explicar que margen cotizado y cobro acumulado son conceptos distintos.

El script `python scripts/demo_http.py --base-url http://localhost:8000` realiza el mismo recorrido y guarda evidencia sin tokens. Requiere identidades configuradas para EVE-C001 y PROV-001. Los importes del ejemplo dependen del CSV original incluido.

Excepciones adicionales

- Cambio a 15 días exactos: cargo cero. A 14 días: 15% del incremento.

- Proveedor que cancela: devolver Backup activo y registrar nueva solicitud con `replaces`.

- 48h: mostrar test determinista del borde sin esperar 48 horas en clase. El temporizador real revisa vencimientos cada minuto.

- Rollback: `test_failed_change_rolls_back_event_proposal_and_notifications` comprueba fallo entre agregados sin dejar notificación.

- Seguridad: token de Cliente no puede registrar pagos ni consultar margen. Otro Cliente no accede al evento.

Preguntas para la defensa

¿Por qué DDD? Las reglas combinan estado, dinero y tiempo. Event conserva pagos/contrato/cambios; Proposal conserva historial; SupplierRequest tiene un plazo independiente.

¿Dónde están AND, OR y NOT? CAN_SEND combina activo, detalle y autorización. CAN_CLOSE niega deuda o información faltante. Hay pruebas de cada combinación.

¿Cómo evitan errores monetarios? Decimal y redondeo ROUND_HALF_UP a centavos. JSON y DB conservan importes como texto. Nunca se calcula dinero con float.

¿Qué sucede a 15 días o a 48h exactas? Día 15 no genera cargo por BR03/H2. A 48h ya no se acepta respuesta por el límite estricto de U3/BPMN.

¿Qué protege la transacción? Agregados, costo/reserva y eventos de dominio relacionados. Si algo falla, rollback. Un update condicionado por versión evita perder cambios de otra transacción.

¿Qué cambió frente a U3? SQLAlchemy Core con snapshots JSON, un servicio coordinador, vistas sencillas y avisos internos persistidos. Se implementaron extensiones de adicionales y cierre. Los ADR documentan costos y límites.

¿Son reales las métricas? Se generan con pytest-cov y se adjunta JUnit/log/HTML. Las metas comerciales del BRD no se presentan como mejoras medidas.

¿Qué falta verificar fuera de este entorno? Ejecución de Compose/PostgreSQL, Actions remoto y despliegue público. Se requieren el repositorio y la plataforma reales. No se simulan esas evidencias.

11. Publicación y pendientes verificables

Los archivos Dockerfile, docker-compose.yml, workflow de Actions y render.yaml preparan un entorno reproducible de DB, App y Tests. No hay evidencia de ejecución de contenedores en este entorno ni URL pública del PIA. El repositorio de destino debe indicarlo el equipo. No se modificaron los repositorios usados como referencia.

Antes de la entrega definitiva se requiere: publicar el código en el repositorio del equipo, ejecutar Actions y Compose, configurar la plataforma, probar /health, /docs y /openapi.json en producción y anotar las URLs y SHA reales en README. El equipo presenta la misma versión que conserva en GitHub.

Las variables separan configuración y código. Se generan tokens aleatorios por identidad y credenciales locales sin incluirlos en el ZIP. La app admite PostgreSQL y SQLite. Las notificaciones son internas; el contrato y los pagos se registran mediante referencias, sin cobro bancario real.

12. Conclusiones

La implementación transforma el análisis de Eventos Nova en reglas verificables para propuestas, contratación, proveedores y cambios, y completa las extensiones de adicionales y cierre. La unidad de trabajo conserva coherencia entre importes, versiones y avisos. Las pruebas muestran los límites de anticipo, plazo y recargo, además de rechazos por estado y permisos.

La elección de snapshots JSON reduce complejidad inicial y conserva fronteras de agregado, a cambio de validar referencias en aplicación y planear futuras migraciones. Los tokens por identidad resuelven la demostración de roles, pero no equivalen a una plataforma completa de gestión de cuentas. Estas decisiones están documentadas para una revisión informada.

La preparación técnica está respaldada por evidencia local. La finalización de los criterios remotos depende de publicar y comprobar el sistema en la infraestructura elegida. Los miembros del equipo deben revisar el trabajo asistido, registrar sus aportaciones reales y poder explicar cada decisión durante la defensa.

13. Referencias y procedencia

Equipo 16. (2026, 4 de septiembre). BRD, User Stories, BPMN y Glosario de Eventos Nova. Documentos proporcionados en el ZIP.

Equipo 16. (2026, 17 de octubre). Informe de arquitectura, ADR y matriz de trazabilidad de Eventos Nova. https://github.com/Oxmar99/EventosNova-Equipo-16, commit ca039e6e79fb070a3b9b8c09dfcc20e71ef85a82.

Laboratorio Unidad 4. (2026). TechStore: componentes de negocio, Specifications y API. https://github.com/OmarHub-star/Actividad-Unidad-4, commit b7ff94501a38e0d2b1cc6d63698dfd4d74fb8df0.

Laboratorio Unidad 5. (2026). Pruebas y calidad de componentes. https://github.com/OmarHub-star/Unidad-5-COMPONENTES-DE-L-GICA-DE-NEGOCIOS, commit eb2401519d947a9c8de2dc1d03d116820042c13b.

Guía PIA CLN AD2026. Proyecto final: implementación de una solución de software basada en lógica de negocio. Documento del curso proporcionado.

Paquete 16_Eventos_Nova. Archivos 01_Alumno, seis CSV y advertencia de adaptación de reglas. No se utilizaron soluciones del docente.

Asistencia de IA: preparación realizada con Codex. El registro distingue decisiones de las fuentes, refinamientos implementados y revisiones pendientes del equipo. Véase docs/USO_IA.md.